Evaluating AI Coding Assistants in Software Engineering: Productivity, Reliability, and Governance

A critical narrative review and proposed evaluation framework

Abstract AI coding assistants can accelerate selected programming tasks, yet speed alone is an inadequate measure of engineering value. This paper synthesizes foundational empirical research on code-generation tools, developer productivity, and software security. It distinguishes task-level gains from lifecycle-level outcomes and proposes a reproducible evaluation design that measures correctness, maintainability, security, and developer experience. The paper is a literature-based analysis; it does not report a new experiment.

1. Introduction

Large language models are now used to propose functions, tests, explanations, and refactorings within development environments. Their appeal is straightforward: they can reduce time spent on routine code production. However, software engineering outcomes depend on more than the time required to type a solution. Incorrect assumptions, hidden vulnerabilities, and maintenance costs may shift work from initial implementation to review and remediation.

The central research question is: under what conditions do AI coding assistants improve net software engineering performance without reducing software quality? This review argues that useful evaluation requires both controlled task measurements and longitudinal evidence from real projects. The distinction matters because faster completion of a constrained task need not imply faster delivery of reliable production software.

2. Evidence and theoretical framing

In a controlled study of GitHub Copilot, Peng et al. (2023) reported that participants with access to the assistant completed a specified programming task substantially faster than participants without it. The finding supports the possibility of task-level productivity gains, but it should not be generalized mechanically to complex, collaborative systems. Effects can depend on the task, developers, prompts, and review practices.

Security introduces a second dimension. Pearce et al. (2022) examined security-relevant code generated with Copilot and documented unsafe outputs in several tested scenarios. Their results do not establish that every assistant-generated change is insecure; rather, they demonstrate why plausible-looking code requires independent validation. NIST (2022) frames secure development as an integrated set of practices, not a final inspection step.

Productivity itself is multidimensional. Forsgren et al. (2021) describe the SPACE framework, which covers satisfaction and well-being, performance, activity, communication and collaboration, and efficiency and flow. This framework is relevant because lines of code and accepted suggestions are weak substitutes for developer effectiveness. A tool can increase activity while leaving defect rates, deployment lead time, or team coordination unchanged.

3. A multidimensional evaluation framework

Correctness. Assess task completion against hidden tests, explicit requirements, and edge cases. Record whether code passes on first submission and after permitted revisions. Human reviewers should also inspect whether generated solutions satisfy the intent of the specification rather than merely matching visible examples.

Maintainability. Evaluate comprehensibility, modularity, testability, and consistency with project conventions. Use blinded review rubrics and record follow-up rework, because a compact generated implementation may be difficult to modify. Static metrics such as complexity should supplement, not replace, expert judgment.

Security and reliability. Run dependency and static-analysis checks, security-focused tests, and targeted adversarial cases. Distinguish vulnerabilities introduced by the assistant from vulnerabilities already present in the starter code. Record severity and exploitability rather than only counting warnings.

Human and team outcomes. Measure elapsed time, interruption frequency, perceived workload, review burden, and confidence calibration. Track whether suggestions lead developers to skip verification. At the team level, monitor pull-request cycle time, escaped defects, and time spent maintaining assistant-authored changes.

4. Proposed research design

A useful study would recruit developers with varied experience and randomly assign comparable tasks to assistant-enabled and assistant-disabled conditions. A crossover design, with counterbalanced task order, can reduce between-person variation. Tasks should include greenfield implementation, bug fixing, refactoring, and security-sensitive work; each category tests a different claim about usefulness.

Pre-register the primary outcome as quality-adjusted completion time: time to a solution that passes independent functional and security acceptance criteria. Secondary outcomes should include review time, maintainability ratings, and developer experience. Analyze effects by task type and experience rather than presenting a single universal speed-up percentage. Record model version, configuration, prompts, repository context, and permitted external tools to improve reproducibility.

A complementary field study should follow real pull requests over several months, comparing teams or periods with careful controls for project complexity and learning effects. A mixed-method approach combining repository data, surveys, and interviews can identify mechanisms that task experiments miss, such as improved onboarding or increased reviewer workload.

5. Threats to validity and ethical considerations

Laboratory tasks may lack realistic codebase complexity; volunteers may be more enthusiastic than typical users; and rapid model changes can make findings obsolete. Contamination is another concern when participants in the control condition have habitual exposure to AI tools. Pre-existing skill differences and uneven task difficulty should be controlled where feasible.

Organizational adoption also raises confidentiality, intellectual-property, and accountability questions. Teams should establish policies for sending proprietary code to external services, document human approval responsibilities, and ensure that generated patches receive the same testing and security review as human-authored patches. Audit logs should support accountability without becoming excessive employee surveillance.

6. Discussion and conclusion

AI coding assistants should be evaluated as components of a socio-technical development workflow, not as autonomous producers of correct code. Current evidence supports targeted productivity benefits and identifies concrete quality risks, but does not justify a blanket conclusion that assistants improve every engineering task. The strongest adoption strategy combines bounded use cases, explicit verification gates, developer training, and ongoing measurement of downstream outcomes.

The proposed framework offers a testable route from anecdotal impressions to evidence-based decisions. Future research should report quality-adjusted time, security outcomes, and maintenance costs alongside speed, and should replicate findings across model versions and project types.

References

Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of developer productivity: There is more to it than you think. ACM Queue, 19(1), 20-48. https://doi.org/10.1145/3454122

National Institute of Standards and Technology. (2022). Secure software development framework (SSDF) version 1.1 (NIST SP 800-218). https://doi.org/10.6028/NIST.SP.800-218

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In 2022 IEEE Symposium on Security and Privacy (pp. 754-768). https://doi.org/10.1109/SP46214.2022.9833571

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590. https://arxiv.org/abs/2302.06590